

Exercises for Week 11

1. Job Script Workflows

To begin with, we will warm by just making some simple job scripts. For all exercises in this section, your job scripts must submit to the 'hpc' queue, use 1 core on 1 node and specify a job name, output file and error file.

1. **Autolab** Make a job script that creates a job array with job indices running from 1 to 10.
2. **Autolab** Make a job script that creates a job array with job indices 2, 29, 71, 73 and 127.
3. **Autolab** Make a job script that waits until another job with id 1234567 exits successfully.
4. Assume we have the following output from `bstat` for a job array which is already submitted:

```

1 $ bstat
2 JOBID      USER    QUEUE    JOB_NAME    NALLOC STAT  START_TIME
   ELAPSED
3 21241475   patmjen hpc      array[1]      1 RUN   Apr  9 13:40
   0:00:09
4 21241475   patmjen hpc      array[2]      1 RUN   Apr  9 13:40
   0:00:09
5 21241475   patmjen hpc      array[3]      1 RUN   Apr  9 13:40
   0:00:09
6 21241475   patmjen hpc      array[4]      1 RUN   Apr  9 13:40
   0:00:09
7 21241475   patmjen hpc      array[5]      1 RUN   Apr  9 13:40
   0:00:09

```

Make job script that creates a job array, where each job in the array waits for the corresponding job in the already submitted array to finish successfully.

5. **Autolab** Again, assume we have the following output from 'bstat' for a job array which is already submitted:

```

1 $ bstat
2 JOBID      USER    QUEUE    JOB_NAME    NALLOC STAT  START_TIME
   ELAPSED
3 21241475   patmjen hpc      array[1]      1 RUN   Apr  9 13:40
   0:00:09
4 21241475   patmjen hpc      array[2]      1 RUN   Apr  9 13:40
   0:00:09
5 21241475   patmjen hpc      array[3]      1 RUN   Apr  9 13:40
   0:00:09

```

6	21241475	patmjen	hpc	array[4]	1	RUN	Apr	9	13:40
		0:00:09							
7	21241475	patmjen	hpc	array[5]	1	RUN	Apr	9	13:40
		0:00:09							

Make a job script that waits until all elements of the job array have exited for any reason.

2. Face Colors



In this exercise we will return to the [CelebA](#) dataset of celebrity images as our data source. As a way of summarizing the dataset, we will create a histogram over the colors in the images - more specifically, the [hue](#) values. The images are stored in subfolders at: `/dtu/projects/02613_2024/data/celeba/images`. Each subfolder contains around 1000 images. To speed up the computation, we will process each folder as a separate job (using a job array) and then aggregate the results into a combined histogram as a final job at the end.

Consider the following function, which extracts a histogram from an RGB stored as a (H, W, 3) NumPy array:

```
1 import numpy as np
2 from PIL import Image
3
4 def huehist(image):
5     bins = np.linspace(0, 255, 64 + 1)
6     hsv_image = np.asarray(Image.fromarray(image).convert('HSV'))
7     hue_values = hsv_image[:, :, 0].reshape(-1)
8     hue_hist = np.histogram(hue_values, bins)[0]
9     return hue_hist
```

1. Create a Python program which takes an index i as a command line argument. For the i th folder in `/dtu/projects/02613_2024/data/celeba/images`, the program must then:
 - a) Load every image in the folder. Hint: use [glob.glob](#) or [os.listdir](#) to get the files in a folder.

- b) Compute the hue histogram for each image using the provided function.
 - c) Sum the histograms to get a combined histogram for the folder.
 - d) Save the summed histogram to a NumPy array named `subhist_<i>.npy`, where `<i>` is replaced with the value of the provided index `i`.
2. Create a Python program that loads the saved histogram arrays, sums them to a combined histogram for the entire dataset, plots the histogram using `plt.bar` and finally saves the histogram plot as an image.
 3. Make a job script that submits a job array where each job computes the hue histogram for the `i`th folder using the first Python program.
 4. Make a job script that waits for all jobs in the previous job array to finish successfully and then runs the second Python program to compute and plot the final hue histogram.
 5. Submit your jobs and compute the histogram.